SN Computer Science (2026) 7:57
https://doi.org/10.1007/s42979-025-04637-3

ORIGINAL RESEARCH

LitCovNet: Attention-Based Lightweight Convolutional Network for Covid and Lung Disease Classification from Chest X-ray Images

Md. Yearat Hossain[1] · **Md. Mahbub Hasan Rakib**[1] · **Rashedur M. Rahman**[1]

Received: 27 August 2025 / Accepted: 9 December 2025
© The Author(s), under exclusive licence to Springer Nature Singapore Pte Ltd. 2025

Abstract

Developing computer-aided diagnosis systems that balance accuracy and efficiency remains a challenge in COVID-19 and respiratory disease classification. To develop effective lightweight models, we investigated various optimization strategies, including the convolutional block attention module, depthwise separable convolutions, global average pooling, residual connections, and knowledge distillation. Leveraging these optimization strategies, we introduce two models, LitCovNet 1 and LitCovNet 2, and evaluate them on the COVID-19 Radiography Database and COVID-QU-Ex datasets. LitCovNet 2 achieves 98.78% accuracy on the Radiography Database and 95.64% on COVID-QU-Ex with only 67,349 parameters, while LitCovNet 1 attains 95.70% on COVID-QU-Ex with 131,861 parameters. Despite being 350× smaller than ResNet50, LitCovNet 2 shows comparable accuracy and markedly superior FLOPs, model size, and inference speed. This makes it a perfect candidate for resource-constrained deployment. Benchmarking on multiple edge devices further demonstrates their suitability for resource-constrained deployment. These results establish LitCovNet as an effective lightweight solution for large-scale multiclass COVID-19 and respiratory disease classification.

Keywords Covid-19 classification · X-ray Imaging · Efficient deep learning · Optimized neural networks · Deep learning

Introduction

Coronavirus Disease 2019 (COVID-19), caused by SARS-CoV-2, has created a tremendous worldwide effect with hundreds of millions of cases reported globally. According to the World Health Organization (WHO), as of January 21, 2024, a total of 774,395,593 COVID-19 cases and 7,023,271 deaths had been reported [1]. Although the WHO declared an end to the Public Health Emergency in May 2023 [2], new infections continue to emerge [3], and the virus is expected to continue well into the future through emerging variants [4]. This present risk underlines the continuing need for rapid, accessible, and reliable diagnostic tools.

Deep learning (DL)-based computer-aided diagnosis (CAD) systems have performed well in various medical imaging tasks, including respiratory disease detection [5–7]. Several CNN-based models have demonstrated expert-level accuracy in different domains [8–13]. A number of works have explored the use of DL in COVID-19 diagnosis using chest X-ray and CT imaging [14, 15]. However, despite the progress, a number of challenges remain in deploying AI-driven CAD systems in real-world clinical and resource-constrained environments. Many of such models are relatively heavy in computation, rely on small datasets, focus mostly on binary COVID versus non-COVID classification, or are based on inadequately articulated design intuition, hence hard to generalize and inefficient for practical use.

Even though a lot of research has been done on the effectiveness of DL in the identification and comprehension of medical disorders, developing an AI-powered computer-aided diagnosis system for use in the actual world is still

✉ Rashedur M. Rahman
rashedur.rahman@northsouth.edu

Md. Yearat Hossain
yearat.hossain@northsouth.edu

Md. Mahbub Hasan Rakib
mahbub.rakib@northsouth.edu

[1] Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh

Published online: 29 December 2025

a challenging task. One of the biggest reasons behind this is typically DL models typically require a large amount of computational resources to operate. However, the demand for efficient CAD systems is growing as more and more ubiquitous systems like wearables are getting into our lives, which unlocks the necessity of frequent and rapid implementation of biomedical systems in our regular lives. Many works have been done to optimize models for biomedical applications that deal with optimal optimization of models to develop cost-efficient and faster computer-aided diagnosis systems [16–18]. Several works have also been performed on the optimizations of models that can classify COVID and other conditions from chest X-rays [19–23]. However, almost all of the works either deal with binary classification or are trained on a low amount of data. On top of that, the intuitions behind the approaches are not always clearly mentioned, which can cause various drawbacks in implementing them in real-world use cases. Also, it is hard to develop generalized systems for efficient biomedical applications, as each type of disease and dataset can lead to various results and performances. Hence, it is important to study and experiment with the topic of developing efficient deep learning models for resource-constrained biomedical applications, following the proper methods that can lead to optimal results instead of using common large-scale models.

In this paper, we approach the problem of developing efficient lightweight deep learning models for COVID and other respiratory disease classifications following the best practices and methods of deep learning model optimization. We take the key ideas from various methodologies for developing lightweight, efficient models and end up proposing two novel lightweight architectures named LitCovNet 1 and 2 that achieve competitive results. We start with a very basic CNN for image classification and gradually introduce various techniques to push the model further and further to achieve better performance with lower computational cost. First, we demonstrate the utility of global average pooling (GAP) instead of using Fully Dense Layers in lowering the learnable parameters of the model. Then we introduce the convolutional block attention module (CBAM) [24] to aid the model learn better by utilizing attention mechanisms. CBAM is a lightweight block that helps the model leverage the power of the attention mechanism while gaining only a small amount of learnable parameters. Then, on the architecture end, we introduce depthwise separable convolutions (DSC) [25] to scale the model up for learning better features while gaining a comparable low amount of parameters. We also introduce residual connections between DSC to develop the LitCovNet 2 that achieves even more competitive results compared to the parameters it contains. We have also applied various training techniques that can lead to optimal results for our architecture. All the experiments are done thoroughly, and the motivations behind each approach are discussed in great detail with the proper reproducibility in mind. The model is experimented on two different datasets and achieves competitive results. Lastly, we have used knowledge distillation (KD) to further improve the models and have shown how they should be used and deployed in real-world applications with significant performance gains.

Our work's primary contributions are:

- Various convolution procedures, attention mechanisms, residual connections, and training methodologies are experimented with to create lightweight models.
- Two novel, efficient, lightweight architectures are proposed for classifying respiratory diseases, including COVID-19, from chest X-ray images.
- The models are tested using the most extensive dataset currently available.
- A thorough benchmark and comparison with other models are displayed on both high-end and edge devices with resource constraints.

Related Works

The usage of CAD systems has grown significantly in recent years for diagnosing a wide range of diseases. CAD systems leverage DL techniques to analyze imaging and non-imaging data, especially CNNs, which are becoming dominant due to their superior performance over traditional ML methods. Several works have proven the efficiency of DL in medical imaging for detecting melanoma [26], brain tumor segmentation [27], and cataract detection [28]. These works, taken together, demonstrate the potential of CNNs in a wide array of medical applications.

Although RT-PCR remains the gold standard for COVID-19 diagnosis, its limitations, including low sensitivity and limited accessibility, necessitated the exploration of DL-based alternatives [29, 30]. CNNs on chest radiographs have shown great promise, with several review works summarizing progress in COVID-19 detection [31, 32]. Many CNN-based models have been proposed, including DeTraC [33], attention-based models [34], and multi-class X-ray classifiers [35], all demonstrating strong performance.

The early research on COVID-19 had to rely on small datasets; thus, synthetic data augmentation with the help of GAN was used [36, 37]. However, these synthetic images may introduce domain artifacts [38], which motivated several attempts to develop larger real-world datasets. The COVID-19 Radiography Database [39] and the large-scale COVID-QU-Ex dataset [40] allowed more robust model evaluations. Subsequent studies explored pretrained CNNs, ensemble techniques, and transformer-based architectures

[41–46], reporting high accuracy across binary and multiclass COVID-19 classification tasks.

Many studies have been conducted on COVID-19 detection from X-ray images using state-of-the-art DL methods, transfer learning, and novel approaches; most of the works focused on using complex CNN models. Though CNN models provide great performance they are composed of many layers and millions of parameters such as ResNet50, a CNN model first introduced in 2015 still popular to this date, and is composed of 50 layers and 25.6M parameters [47]. These models are computationally expensive and not viable to deploy on edge devices because of the limited computational power of edge devices. Cheng et al. [48] highlighted the limitations of CNNs on devices with limited resources, such as memory, CPU, bandwidth, and energy. The scientific community is thus becoming more aware of the need for compression approaches that can lower the number of model parameters and their resource requirements. There are different compression methods. Among them are parameter pruning and sharing, low-rank matrix and tensor decomposition, compact convolutional filters, and knowledge distillation. These methods make it possible to deploy CNN models on low-end edge devices [49]. Alabbasy et al. [22] analyzed compression techniques and compared their performance. They proposed a framework that utilized knowledge distillation to deploy the models on edge devices for disease diagnosis. They were able to compress their proposed knowledge distillation-based COVID-19 and Malaria models by 18.4% and 15%, respectively, and also accelerate the response by 6.14× and 5.86×, respectively, while retaining performance. The authors of [21] introduced a lightweight deep learning method for COVID-19 classification using the MobileNetV2 model. It shows equivalent performance to heavyweight models but greatly increases deployment efficiency by lowering memory requirements and computer resource costs. The approach they suggested yielded results of 94.5% overall accuracy, 92.3% sensitivity, 96% specificity, and 5% misclassification rate. Hussein et al. [23] introduced two lightweight CNN models, one for binary classification and the other for multiclass classification, to use on devices with limited resources. Their proposed binary classification model achieved 98.55% accuracy while the multiclass classification model achieved 96.83%.

Though many works have been published on COVID classification, the majority of them are done as binary classification or utilize small datasets, resulting in a lack of robustness in the models. Also, another common practice is utilizing very large models without considering the requirements of the desired tasks, which makes the trained models unusable in resource-constrained scenarios. In our work, we addressed all these study gaps in great detail and provided future directions toward multi-class COVID and other respiratory diseases from chest X-ray images using extremely lightweight CNN models.

Methodology

In this section, we provided a detailed discussion of all the components of our architecture and the dataset. First, we described the datasets used in our experiment. Then we discussed CNN and various other concepts related to it that helped us construct the proposed architecture. We have discussed Convolution Neural Networks (CNN), Pooling and Global Average Pooling (GAP), Convolutional Block Attention Module (CBAM), and Depthwise Separable Convolution (DSC). Then we discussed the details of our proposed lightweight models. Finally, we discussed the method of Knowledge Distillation along with the metrics used in our experiments. An illustration of the overall proposed LitCovNet framework is shown in Fig. 1.

Datasets

For any image classification problem, datasets are the fundamental component, as they are the most crucial element for training, testing, and validating models. The performance of DL models is highly dependent on the quality and size of the dataset. Low-quality data leads to inferior performance, and a small dataset leads to training overfitting, where the model performs well on the training set but

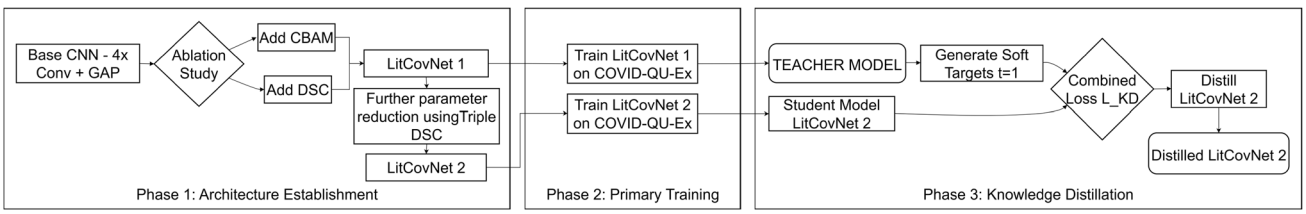

Fig. 1 Overall pipeline of the proposed LitCovNet framework, consisting of three phases: In Phase 1, a base CNN is refined through ablation studies by adding CBAM and DSC, producing LitCovNet-1 and the parameter-efficient LitCovNet-2. Phase 2 involves independently training both variants on the COVID-QU-Ex dataset. In Phase 3, knowledge distillation is applied, with LitCovNet-1 as the teacher generating soft targets to train LitCovNet-2, resulting in a distilled and more accurate model

Table 1 Sample distribution of the COVID-19 Radiography Database used in our experiments

Dataset	COVID-19	Lung opacity	Viral pneumonia	Normal	Total samples
COVID-19 radiography database	3616	6012	1345	10,192	21,165

The dataset contains four classes, including COVID-19, Lung Opacity, Viral Pneumonia, and Normal, with their corresponding sample counts

Table 2 Sample distribution of the COVID-QU-Ex dataset

Dataset	COVID-19	Non-COVID infections	Normal	Total samples
COVID-QU-Ex	11,956	11,263	10,701	33,920

The dataset consists of three diagnostic categories, including COVID-19, Non-COVID Infections, and Normal, with their corresponding sample counts

underperforms in the unseen testing set, indicating poor generalization ability of the model. The quality and size of the dataset ensure good performance and generalization of the DL model. In this research, for COVID-19 disease identification purposes, we have used two chest X-ray datasets: COVID-19-Radiography-Database [39] and COVID-QU-EX [50]. The COVID-19 Radiography Database dataset is one of the largest publicly available COVID-19 datasets, containing 21,165 samples in total. It consists of four classes: Covid, Lung_Opacity, Normal, and Viral Pneumonia. The COVID class contains 3616 COVID-positive chest X-ray images, and Lung_Opacity contains 6012 images of non-COVID infections, such as bacterial infections. The Normal class contains 10,192 images of healthy lungs, and the Viral Pneumonia class contains 1345 images of infected lungs from viral pneumonia. Following the works of others, we only worked on COVID, Viral Pneumonia, and Normal class classification problems, skipping the Lung_Opacity. The COVID-QU-Ex dataset is an extended version consisting of 33,920 chest X-ray images, created by combining the COVID-19 Radiography Database [39] and QaTa-Cov19 dataset [41]. This dataset consists of three classes: COVID-19, Non-COVID infections (viral or bacterial pneumonia), and Normal images. Among 33,920 images, 11,956 are COVID-19-positive instances, 11,263 are Non-COVID-infectious, and 10,701 are Normal images. The image size of both datasets is 299×299. The details of the datasets are shown in Tables 1 and 2, and the sample distribution along with the count is visualized in Fig. 2. Both datasets are accessible to the public on Kaggle (https://www.kaggle.com/). The COVID-19 Radiography Database was published in this research [39] and is accessible via https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database, whereas the COVID-QU-EX dataset was published in [50] and is available via https://www.kaggle.com/datasets/anasmohammedtahir/covidqu. The authors in [39, 50] have guaranteed that the patient information is de-identified and anonymous for both datasets. Ethical approval is not required for the current research, as the dataset is publicly available through other studies.

Convolutional Neural Network

Convolutional neural networks (CNNs) are robust neural network architectures primarily used in deep learning, created especially for signal, image, and video processing. [51] first introduced CNNs in 1998. Later, [52] introduced AlexNet in 2012, which brought a quantum leap in computer vision. CNNs have demonstrated impressive performance across a range of applications, particularly in computer vision tasks including object detection, image segmentation,

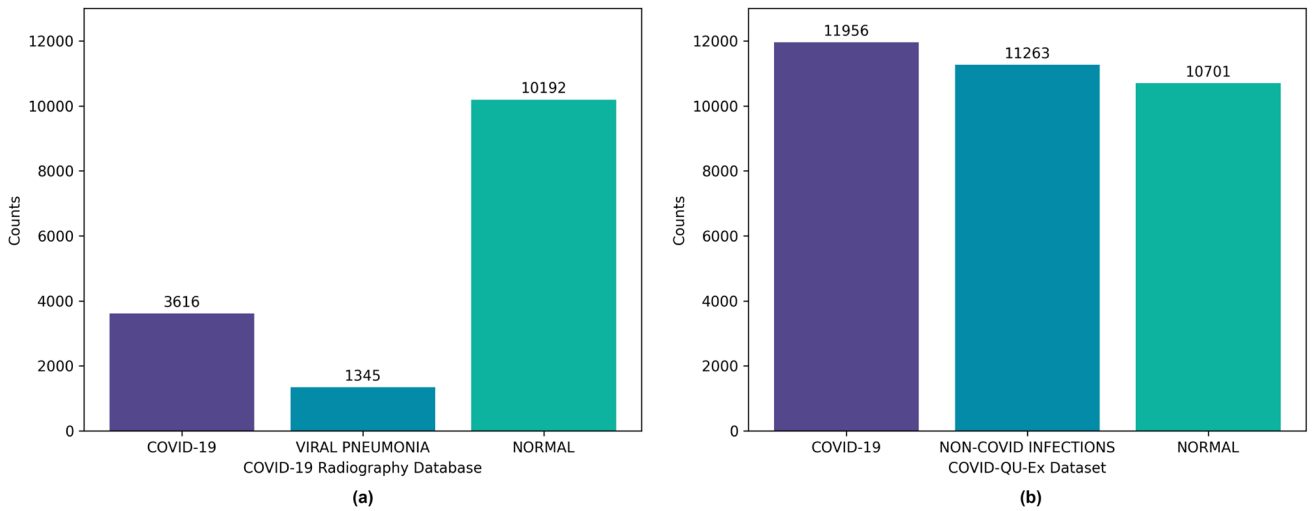

Fig. 2 Sample distribution and the count of images of the datasets used in our research

and image classification. They excel at recognizing patterns and extracting features from data. CNNs are composed of multiple layers. Mainly, they consist of three types of layers: the convolutional layer, the pooling layer, and the fully connected layer. CNNs learn hierarchical visual representations by applying cascaded convolutional filters that capture local spatial correlations and progressively transform raw pixel data into high-level semantic features. A standard convolutional layer performs linear filtering followed by nonlinear activation, enabling the extraction of complex patterns, while pooling or strided operations reduce spatial dimensionality and enhance translation invariance.

Despite their effectiveness in medical image analysis, conventional convolutions impose significant computational and memory costs and do not inherently model inter-channel dependencies or emphasize salient spatial regions. These limitations motivate the integration of architectural refinements that improve feature selectivity and efficiency. In this work, we build upon the CNN backbone by incorporating three complementary components: the Convolutional Block Attention Module (CBAM) to adaptively recalibrate channel and spatial-wise responses, Depthwise Separable Convolutions (DSC) to decouple spatial and channel filtering for substantial reduction in FLOPs and parameters, and Global Average Pooling (GAP) to generate compact global descriptors without fully connected layers. Together, these modules enhance representational capacity while maintaining the computational efficiency required for deployment on resource-constrained edge devices.

Global Average Pooling

GAP is a parameter-efficient technique commonly used to replace traditional fully connected layers at the end of a CNN, thereby significantly reducing model complexity and computational cost [53]. Unlike conventional pooling operations, GAP produces a single value for each feature map by averaging all spatial activations. For an input feature map F_i with dimensions $H \times W \times C$, the GAP operation generates an output vector V of size C using Eq. 1, where each element V_c represents the mean activation of the c-th channel. This operation yields a substantial reduction in parameters compared to flattening the feature maps and feeding them into a dense layer, making GAP preferable in scenarios where model efficiency is critical. Therefore, we adopt GAP instead of a flatten layer.

$$V_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} F_i(i,j) \tag{1}$$

Convolutional Block Attention Module

CBAM is a lightweight and effective attention mechanism that enhances feature representations by sequentially refining them along both channel and spatial dimensions [24]. The Channel Attention Module (CAM) M_c, defined in Eq. 2, assigns importance weights to each channel by processing average-pooled and max-pooled descriptors through a shared Multi-Layer Perceptron (MLP), followed by a sigmoid activation. The Spatial Attention Module (SAM) M_s, formulated in Eq. 3, highlights key spatial regions by concatenating channel-refined average-pooled and max-pooled feature maps, applying a convolution $f^{7\times7}$, and then a sigmoid function. The final refined output is obtained by sequentially applying both attention maps to the input feature map, as shown in Eq. 4.

$$M_c(F) = \sigma(\mathrm{MLP}(\mathrm{AvgPool}(F)) + \mathrm{MLP}(\mathrm{MaxPool}(F))). \tag{2}$$

$$M_s(F) = \sigma\left(f^{7\times7}\left([\mathrm{AvgPool}(F);\ \mathrm{MaxPool}(F)]\right)\right). \tag{3}$$

$$F_{\mathrm{out}} = F \otimes M_c(F) \otimes M_s(F). \tag{4}$$

Depthwise Separable Convolution

DSC, first introduced in [25], is widely employed in applications where computational efficiency is crucial, particularly in embedded and mobile platforms. Due to its lightweight nature, DSC has been incorporated into several influential architectures such as, Xception [54], and MobileNets [55], with the latter using DSC as its primary convolutional unit. Compared to standard convolution, DSC drastically reduces parameter count and computational cost by decomposing the operation into: (i) a depthwise convolution, formulated in Eq. 5, applies a single spatial filter to each input channel, and (ii) a pointwise convolution, defined in Eq. 6, aggregates the depthwise outputs through a 1×1 kernel.

$$F_c^{\mathrm{dw}} = W_c^{\mathrm{dw}} * X_c, \tag{5}$$

$$F_k^{\mathrm{pw}} = \sum_c W_{kc}^{\mathrm{pw}} \cdot F_c^{\mathrm{dw}}, \tag{6}$$

where X_c denotes the c-th input channel, W_c^{dw} is the depthwise kernel applied to that channel, and W_{kc}^{pw} represents the pointwise weight mapping input channel c to output channel k. For an input feature map of size $D_F \times D_F \times C_{\mathrm{in}}$ with a depthwise kernel of size $D_K \times D_K$, the computational savings relative to standard convolution are quantified using Eq. 7.

$$\frac{DSC\ Multiplications}{Standard\ Multiplications} \approx \frac{1}{C_{out}} + \frac{1}{D_K^2}. \quad (7)$$

This considerable reduction in operations has made DSC a fundamental component in high-performance models designed for resource-limited environments.

Residual Connection

Residual connections, or skip connections, introduced by [47], enhance training stability and mitigate gradient degradation by adding the input x of a layer to its output $F(x)$, as formulated in Eq. 8. This element-wise addition provides a direct pathway for both features and gradients to bypass intermediate transformations, thereby improving feature propagation, reducing information loss, and enabling the effective training of deeper and more robust networks.

$$y = F(x) + x. \quad (8)$$

Proposed Architecture

In this section we explain the design of our proposed models in great detail. In our work, we proposed 2 CNNs named as LitCovNet 1 and LitCovNet 2, where the 2nd one contains almost half the size of parameters compared to the 1st one. Though in this section we discuss the whole architectures of the 2 models, how we concluded these particular design decisions is greatly discussed in "Experiments and Results" section Experiments and Results.

LitCovNet 1

For better representation and ease of understanding, we present the core components of the models as blocks. A block may contain various layers that perform various operations like convolution, batch normalization, maxpool, activation, dropout, etc. LitCovNet 1 contains 4 main blocks and the final block, which is similar for both models. A detail of the architecture of LitCovNet 1 is shown in Fig. 3. The first block of LitCovNet 1 contains components in the following order: Conv2D, CBAM, ReLU activation, BatchNormalization, and MaxPool2D. Only in the 1st block of both of models, we apply CBAM, and the activation is applied just after the CBAM layer. The first block outputs a feature map with a depth of 32. Then the 2nd and 3rd blocks perform the similar Conv2D, Batch Normalization, and MaxPool2D operation, where the 2nd block produces a feature map of depth size 64 and the 3rd block produces a feature map of depth size 128. All the Conv2D layers in these aforementioned blocks contain 3×3-sized kernels and paddings are kept 'same', and the activation function used is ReLU. The filter/channel count just doubled in size, which corresponds to the depth size of the created feature maps. On the 4th block, we implement a Depth-Wise Separable convolution operation with a BatchNormalization. To implement the DSC, first, we performed a DepthWiseConv2D operation on the output of the 3rd block and then applied a point-wise convolution operation with Conv2D. The output of the 4th block produces a feature map that contains a depth size of 256. The final block is a combination of Global Average Pooling, Dropout, and BatchNormalization that maps the output of the features to a softmax activation, which eventually gives the final prediction result. Upon compiling the model, the total parameters of the model become

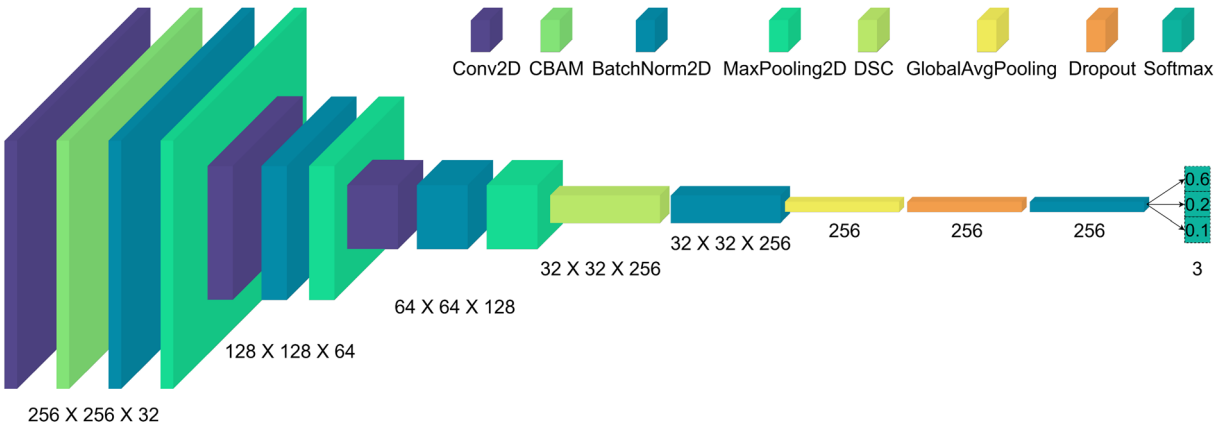

Fig. 3 Architecture of the proposed LitCovNet 1 model. The network begins with consecutive Conv2D, BatchNorm, and MaxPooling layers to extract low-level features, followed by a CBAM module to enhance channel and spatial attention. As the spatial resolution decreases and channel depth increases, the model incorporates DSC to reduce computation, enabling efficient feature learning, then applies GAP and dropout for compact representation and regularization. A final Softmax layer outputs probabilities for the three target classes

131,861, where 130,389 are trainable and 1472 parameters are non-trainable.

LitCovNet 2

LitCovNet 2 contains the exact same 1st, 2nd, and last block of LitCovNet 1. However, the 3rd and 4th block of LitCovNet 1 is replaced with a triple residual Depth-Wise Separable block, which acts as the 3rd block in LitCovNet 2. One of the key factors that contributes to the total parameter count of a model is the number of filters in a convolution operation. However, this parameter count, along with the amount of computation necessary to perform a standard 2D convolution operation can be reduced by implementing DSC. However, the increased number of filters is required by a CNN to learn deeper features better which results in optimal performance. To tackle this, we introduce a complex block that contains 3 DSCs, where all contain filter counts of 128, and use a residual connection to regain some of the lost features throughout the entire operation. The block itself can be thought of as made of 3 sub-blocks. A visual representation of this entire triple DSC residual block is shown in Fig. 4. The 1st sub-block performs a DSC operation and then applies a MaxPool2D on the feature map. This max-pooled feature map is stored, which is later added to the feature maps produced by the 2nd sub-block. After the max pooling, a batch normalization is applied, and then the feature is passed to the 2nd sub-block, where a similar DSC is performed just before performing an element-wise summation with the previously stored feature map. Then a MaxPool2D is applied, and again the feature map is stored for another element-wise summation operation to be performed in the last sub-block. The last 3rd or last sub-block performs the exactly similar to operation as the 2nd sub-block, where the feature map of the previous sub-block is added to the feature map produced by the DSC operation, except this does not perform max pooling and just performs batch normalization on the feature map. All the DSC layers of the 3rd block perform a similar operation, which is producing a feature map of size 128 where the kernel size is set to 3×3, padding is kept the same, and ReLU is used as the activation function. However, the dimension of the feature map changes inside the block as we perform max pooling operations. It was observed that features that are not batch-normalized performed better when added to another feature map instead of adding features that have passed through a batch normalization operation. Hence, the feature map for the residual connection and operation was obtained and performed before the batch normalization operation. Finally, the feature map is passed to the last block of LitCovNet 2, which is exactly similar to LitCovNet 1. After the compilation of the model, the total number of parameters becomes 67,349, where 66,133 are trainable and 1216 are non-trainable parameters. The entire architecture of the LitCovNet 2 is shown in Fig. 5.

Knowledge Distillation

KD [56] is a model compression technique that enables a compact student network to learn from a high-capacity teacher network. Instead of training the student solely with hard one-hot labels, KD leverages the teacher's soft probability distribution, which contains richer information about the relative similarities between classes. These soft targets are generated using Eq. 9.

$$p_i^{(t)} = \frac{e^{z_i/t}}{\sum_j e^{z_j/t}}, \tag{9}$$

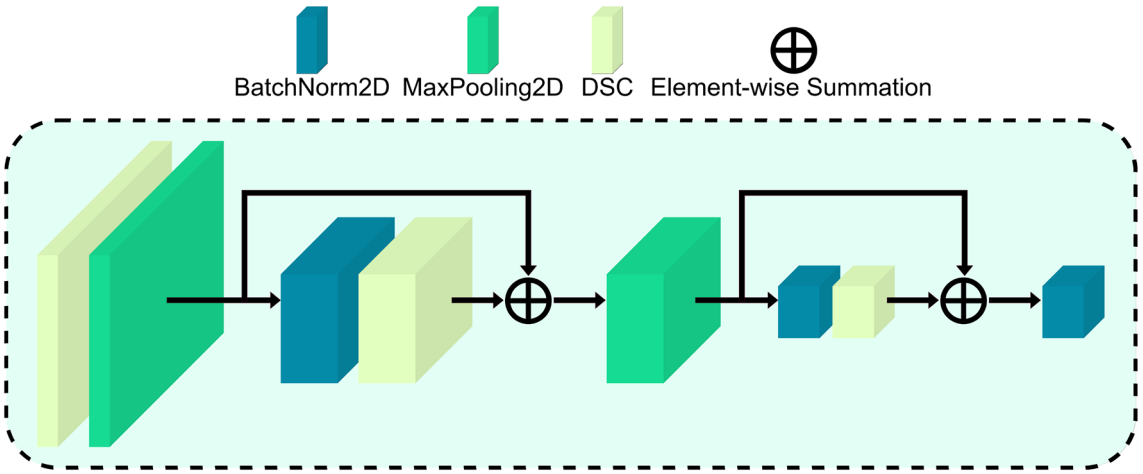

Fig. 4 Internal view of the proposed triple depthwise separable residual block. The block applies three consecutive DSC with BatchNorm and MaxPooling, while two skip connections perform element-wise summation to preserve the original feature information. This design enables the block to retain fine-grained details from earlier layers while simultaneously learning new, enriched feature representations

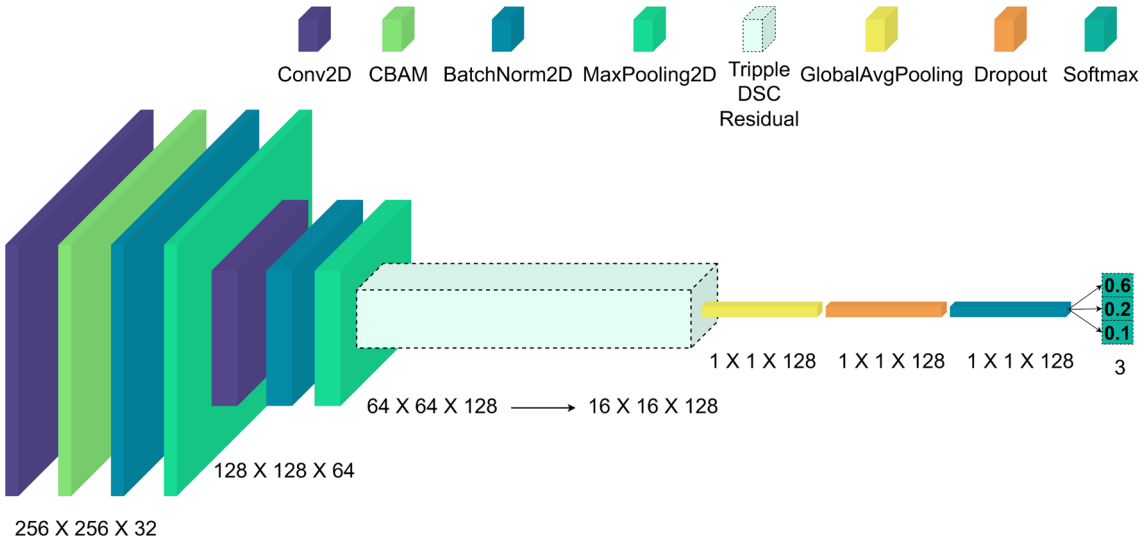

Fig. 5 Architecture of the proposed LitCovNet 2 model. This variant follows the same initial feature extraction pipeline as LitCovNet 1. The main distinction is the integration of a triple depthwise separable convolution residual block, which processes intermediate feature maps from $64\times64\times128$ down to $16\times16\times128$ while preserving spatial details through residual summation

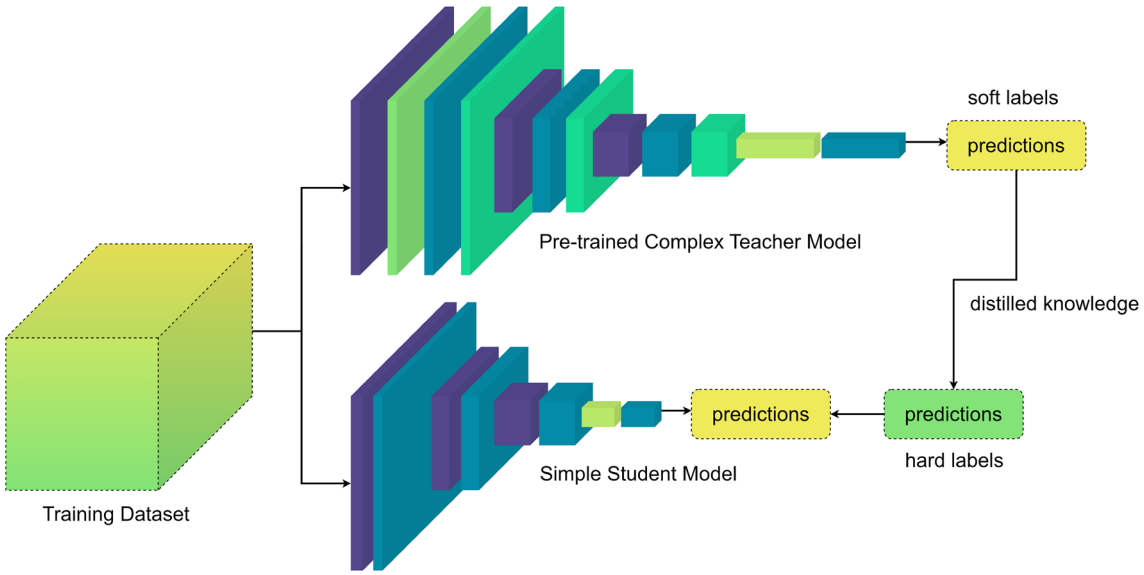

Fig. 6 A high-level overview of the KD process, where a pre-trained, high-capacity teacher model transfers its learned representations to a lightweight student model. During training, the teacher generates soft-label predictions that capture rich class relationships. These soft labels, combined with the ground-truth hard labels, guide the student model to learn more effectively, enabling it to approach the teacher's performance while maintaining significantly lower complexity

where z_i is the logit for class i. These soft targets allow the student to learn subtle relationships between classes, improving generalization, especially with limited training data. The training of the student model is guided by a combined weighted loss function ($\mathcal{L}_{\text{KD}}$), defined in Eq. 10, blends the student loss and a distillation loss using a balancing hyperparameter α.

$$\mathcal{L}_{\text{KD}} = \alpha\mathcal{L}_{\text{student}} + (1-\alpha)\mathcal{L}_{\text{distill}}, \tag{10}$$

where $\alpha \in [0,1]$ balances the conventional cross-entropy loss with the distillation loss derived from the teacher's soft labels. The temperature t controls the smoothness of the teacher's output distribution: a higher t produces softer probabilities, emphasizing inter-class similarities, which in turn enables more effective knowledge transfer and improved student performance. The overall workflow of the KD process is depicted in Fig. 6.

Metric

Accuracy

A model's performance is evaluated based on accuracy. Accuracy is a model's capacity to classify or predict results accurately. It serves as an indicator of the model's effectiveness on a particular dataset. An increased accuracy score signifies that the model yields an increased number of accurate predictions in contrast to inaccurate ones. It suggests that the model is functioning effectively in terms of accurately classifying samples from all classes included in the dataset. On the other hand, a lower accuracy score indicates that the model predicts more things incorrectly than correctly. It suggests that there may be room for improvement in the model's performance with regard to classification accuracy. Accuracy is calculated as the ratio of the number of correct predictions to the total number of predictions. The equation of accuracy is given in Eq. 11.

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Number\ of\ Predictions} \tag{11}$$

Experimental Setup

We conducted all of our experiments on a desktop computer that contains a 3.40 GHz Intel Core i7-13700K CPU, 32 GB DDR5 memory, and a 24 GB Nvidia 4090 GPU. The computer uses solid-state drives as permanent memory and Windows 11 Pro as its operating system. On the software end, we used Tensorflow (2.10) as the machine learning framework, and to ensure the reproducibility of the results, we enabled op determinism, and a global random seed was set to 42. Op determinism is enabled by calling the function *'tf.config.experimental.enable_op_determinism()'*, which ensures reproducibility on the hardware end and always produces the same result given the same input and setup. This helps us to debug and assess our model development properly. However, this may cause slower training due to the restrictions put on the hardware. Also, a detailed overview of the training setup of all the experiments is given in Table 3. For preprocessing, all images are converted to grayscale and rescaled to 256×256 pixels. Pixel values are then normalized by dividing by 255 to bring them into the range [0, 1]. No data augmentation techniques (such as rotation, shifting, or flipping) are applied for training.

Experiments and Results

In this section, we provide a comprehensive analysis of how we built the architectures and trained them to achieve the optimal result. First, we begin with the small dataset and experiment with various architectural designs to figure out the optimal design. For this, we utilize a small image resolution and a relatively basic training setup so that we achieve the results faster.

Then we scale up our experiment with a larger balanced dataset to test if the constructed model is generalized from the aspect of performance and adaptability. Here we use higher-resolution images and employ a slightly different training strategy to achieve the best results without changing anything from the previously designed architecture. After observing the performance patterns of the previous two sections, in the third subsection, we introduce another model that is comparatively lighter than the first model and achieves competitive performance compared to the shrunken parameter count. Also, we perform knowledge distillation to improve the performance of the models without introducing any more parameters.

Throughout the whole section, we ran all the experiments following an ablation study manner and provided all the necessary tables and figures wherever required.

Table 3 Comprehensive overview of all training configurations used in our experimental pipeline

Training stage	Sec.	Dataset	Dim.	Batch	Epochs	Optimizer setup
Architecture establishment	5.1	COVID-19 Radiography	112 X 112 X 1	32	100	Adam (LR = 0.001)
LitCovNet 1	5.2	COVID-QU-Ex	256 X 256 X 1	128	100	Adam + ReduceLROnPlateau (patience = 8, LR 0.01→1e−7)
LitCovNet 2	5.3	COVID-19 Radiography	256 X 256 X 1	64	300	Adam (LR = 0.001)
LitCovNet 2	5.3	COVID-QU-Ex	256 X 256 X 1	128	100	Adam + ReduceLROnPlateau (patience = 8, LR 0.01→1e−7)
KD LitCovNet 1	5.4	COVID-QU-Ex	256 X 256 X 1	64	100	Adam + ReduceLROnPlateau (patience = 10, LR 0.001→1e−7)
KD LitCovNet 2	5.4	COVID-QU-Ex	256 X 256 X 1	64	100	Adam + ReduceLROnPlateau (patience = 10, LR 0.001→1e−7)

Each row specifies the training stage, referenced section in the paper, dataset used, input image size, batch size, number of epochs, and the optimizer settings

Establishment of the Architecture

First, we established the construction of our proposed architecture utilizing the COVID-19-Radiography Database. This dataset was used in this case as it is smaller in size compared to the other dataset, and this helped to train models faster. First, we constructed the base architecture, which only contains 4 repeating convolution blocks and a GAP layer with dropout at the end. This GAP is a replacement for the dense layer which is widely used in image classification models. The base architecture consists of 391,555 parameters only. For all the experiments done with the COVID-19-Radiography Database, we used grayscale images resized to 112×112 in dimension and normalized them to 0 to 1. We have used a batch size of 32 and trained the model for 100 epochs. We used Adam as the optimizer and CategoricalCrossEntropy as the loss function, where accuracy is the metric of the model. Also as the dataset is not properly balanced, a class weight of {0: 1.3969, 1: 0.4956, 2: 3.7542} was used following the work [23]. The class weight was implemented using the method of study [57]. The weights correspond to the classes 'covid', 'normal', and 'viral-pneumonia'. The dataset was first split into 80:20 for training and testing purposes, and then the split training set was split again into 80:20 for training and a validation set. This approach is also followed by the paper [23]. During the training, the best model is saved where the minimum loss with maximum accuracy on the validation set is achieved. As the training finished, the base model achieved an accuracy score of 98.39 on validation and 97.92 on the test set. The base model alone beats the score of 96.83 on the test set that was reported at work [23] with much lower parameters. All the training and testing results, along with the parameters of the models is shown in Table 4.

Then we introduced the Convolutional Block Attention Module or CBAM inside our architecture. The CBAM block was implemented just after the Conv2D operation and before the batch normalization and max-pooling of the repeating convolution blocks. Our base architecture contained 4 main blocks, and we have tried all possible permutations of inserting CBAM and experimented with it. We concluded that the CBAM block only works on the first block and yields a better result than the base model. When the CBAM is included in the first block, the total parameter count slightly increases to 392,725. However, upon training the model, an accuracy score of 98.43 is achieved on the validation set, and 98.02 is achieved on the test set. This score beats the previous score achieved by the base model. Here the effectiveness of CBAM is noticeable, although it only worked on the 1st block, leading to the conclusion that in our experiment, the CBAM block has its efficacy when the features contain higher spatial dimensions. As we went deeper into the base model's architecture, we increased the number of channels at each block which introduces 256 channels on the 4th block. This increased number of channels introduces a significant number of parameters in the model hence we swapped the 4th block's Conv2D layer with a Depthwise Separable Convolution and removed the batch normalization layer from the base model (excluding CBAM). This time, the model's parameters were significantly reduced to 130,691. The impact of this parameter reduction is noticed when the model is trained and achieves accuracy scores of 97.73% and 97.43% on the validation and test set. These scores are lower than the scores achieved by the base model where standard convolution operation is used instead of DSC. Afterward, we introduce CBAM on the 1st block again on the model that contains DSC on the 4th block. This time, the parameter of the model again slightly increases to 131,861. However, after training the model, accuracy scores of 98.47% and 98.15% are achieved, which is the highest set of scores among all the experiments. Here, again, the effectiveness of the CBAM module is prominent, which leads to the highest performance of the model while bearing a significantly reduced parameter count. We named the final model LitCovNet 1.

Table 4 Performance comparison of baseline and enhanced model variants during the architecture establishment phase

Method	Validation Acc	Test Acc	Params
LW-CNN [23]	97.20	96.83	592,929
Base model	98.39	97.92	391,555
Base model + CBAM	98.43	98.02	392,725
Base model + DepthWiseSeparable	97.73	97.43	130,691
Base model + DepthWiseSeparable + CBAM	98.47	98.15	131,861

The table shows how integrating CBAM and DSC affects validation accuracy, test accuracy, and parameter count relative to the LW-CNN baseline

Experimentation with Larger Dataset

As we established the design of LitCovNet 1, we moved towards experimenting with it with a much larger and balanced dataset. This time, we utilized the COVID-QU-EX dataset. The dataset is available as split into train, validation, and test by the creators hence we did not manually split them. However, we changed our experimental setup a bit for this experiment. As this dataset is quite large compared to the previous dataset, we have increased the image size and batch size to 256 × 256 and 128, respectively. Here, we performed an online image augmentation that creates a random variation of the training set so that the model can generalize better. This also helps to avoid overfitting, where the model learns to classify the training set well and struggles

Table 5 Performance of LitCovNet 1 on COVID-QU-EX dataset with and without CBAM. Though CBAM introduces 1170 parameters, the performance gain is almost 0.50%

Model	Validation Acc	Test Acc	Params
LitCovNet 1	93.69	95.40	131,861
LitCovNet 1 w/o CBAM	93.59	94.93	130,691

to classify the validation set properly. For the augmentation, we applied a horizontal flip and a brightness shift ranging from 0.9 to 1.1 on the training set. Online augmentation does not necessarily increase the total count of training data; instead, it creates variation of the dataset during training time and helps the model to achieve robustness. This augmentation setup is used for all further training in our experiments. We have also employed a learning rate scheduler on the optimizer so that the model learns optimally. All these are hyperparameters, and tuning them properly requires extensive experiments. We ran several experiments to figure out the best possible setup of these hyperparameters and selected the values accordingly. We have utilized the ReduceLROnPlateau as the learning rate scheduler. The LR scheduler's job is to monitor the validation loss and change the learning rate accordingly. In our experiments, the scheduler checks if the validation loss gets stuck on a plateau. A patience of 8 epochs is used to observe if the validation loss gets lower or not. We begin with a high learning rate of 0.01, and if the validation loss does not improve within 8 epochs, the scheduler lowers the learning rate by a factor of 0.1, while the minimum learning rate is defined as 1e-7. The number of epochs was kept at 100 as the training always converged within 100 epochs.

With all these setups, the LitCovNet 1 model, comprising the base model, CBAM, along with DSC, achieved a maximum accuracy score of 93.69% on the validation set and 95.40% on the test set. The performance of LitCovNet 1 is displayed in Table 5 along with its performance without the CBAM block. Through all of our experiments performed on the COVID-QU-EX dataset, it was observed that the performance on the unseen test set was always relatively better than the validation set. However, this is not an issue, and it does not occur due to the architecture of the model. The dataset itself contains samples in the test set that are relatively easier for a trained model to classify. This also means that the samples of the training set resemble the samples of the test set relatively better than the validation set.

Slimming Down the Architecture

Finally, inspired by the performance of LitCovNet, we experiment further to shrink down the parameters of the model that yields competitive performance with fewer parameters than LitCovNet 1. The effectiveness of CBAM and DSC is already demonstrated in LitCovNet 1. However, the number of channels introduced in the Conv block that contains 128 filters contributes significantly to the increased parameter count. Also, the following DSC block with 256 filters increases the parameters. Although reducing the filters reduces the total parameters, it also comes with a performance trade-off. To tackle this, we experimented with various design patterns and introduced a 3-step DSC with residual connections in between. For this, we dropped the last Conv and DSC block and swapped them with 3 consecutive DSC blocks that contain a similar number of filter count, which is 128. Instead of using the trailing batch norm and maxpool like the conv layers, we use maxpool first here, just before the batch norm layer. This was done as the features collected before the batchnorm operation produced better results when skip connections were established. In this architecture, batch norm layers overregularize the feature map, which doesn't help the residual connection to achieve a better result; hence, the skip connections are collected before the batchnorm layer. The first feature is collected from the 1st DSC block, and concatenation is performed on the 2nd DSC block before the maxpool is applied. This helps to properly perform the element-wise summation operation of similar-sized feature maps without introducing any new type of operation. Then another skip connection is acquired after the maxpool and pointwise summations are performed just before the final batch norm of the 3rd DSC layer. The 3rd layer does not contain any maxpool, just like the DSC layer of the LitCovNet 1.

Table 6 Validation and test accuracy of the lightweight LitCovNet 2 model on two benchmark chest X-ray datasets, COVID-19-Radiography and COVID-QU-Ex, demonstrating its generalization ability across different data sources

Dataset	Validation Acc	Test Acc
COVID-19-radiography	99.26	98.78
COVID-QU-Ex	93.26	95.18

As we only used 3 layers at the end, which are all DSC and contain a filter size of 128 only, the parameter count significantly shrinks down to 67,349. We named this model LitCovNet 2 and trained it with the datasets used in our previous experiments. We used the same training setup for training the Covid-Qu-Ex dataset. However, for training the COVID-19-Radiography dataset, we used an image resolution of 256×256 and a batch size of 64. We trained this model with this dataset for 300 epochs, as the model kept learning for a longer period. The training results of LitCovNet 2 are shown in Table 6. Here, LitCovNet 2, despite being nearly half in size of LitCovNet 1, beats the previous scores achieved on the COVID-19-Radiography Dataset by LitCovNet 1 during the establishment of the design of architecture. Here, LitCovNet 2 achieved a validation accuracy of 99.26% compared to the previous score of 98.47%, and on the testing, it achieved 98.78% compared

to the previous test score of 98.15%. However, this result boost is not necessarily completely reliant on the architecture itself. During training LitCovNet 2 on the Radiography dataset, we increased the image resolution from 112×112 to 256×256, which provided better information to the model and applied image augmentation to prevent overfitting, while the architecture itself required longer training to reach its convergence.

It is crucial for a model to learn complex patterns in deeper layers, and this operation is mainly performed by the increased number of convolution filters introduced in the deeper layers of a CNN. However, as we used residual connections along with 3 sequential DSC blocks containing 128 filters, the learned features were preserved better, which helped to achieve better results. The effectiveness of this residual design along with CBAM in LitCovNet 2 is shown in Tables 7 and 8. It can be seen that the CBAM and residual connections help the model greatly to produce better results. However, removing the CBAM block significantly impacts the model compared to removing the residual design. However, keeping the residual design active along with CBAM produces the best results. Furthermore, the residual connection does not introduce any parameters in the model, yet helps the triple DSC blocks to achieve greater performance with a very low amount of parameters.

To further ensure the reliability and robustness of the reported results, we incorporated an additional evaluation protocol. Because the models were trained with full determinism enabled-controlling all random seeds, enforcing deterministic GPU kernels, and disabling stochastic operations during inference-the output of the network is identical across repeated test runs on the same dataset. Consequently, the conventional approach of performing multiple independent runs to estimate variance or confidence intervals becomes ineffective, as it yields no measurable dispersion. To address this limitation while maintaining methodological rigor, we constructed four distinct, non-overlapping evaluation sets using clinically plausible image transformations: (i) the original test set, (ii) a horizontally flipped version of the test set, (iii) the original validation set, and (iv) a horizontally flipped version of the validation set. Horizontal flipping was selected because left-right mirroring can naturally arise in radiographic workflows due to acquisition orientation or post-processing operations, whereas vertical inversions or rotational distortions were excluded since they do not reflect valid radiological conditions and would instead introduce artificial degradation. Each of these four test sets was inferred exactly once, thereby producing four deterministic yet independent performance estimates that reflect the model's behavior under realistic distributional shifts. These scores were subsequently aggregated to compute the mean accuracy, standard deviation, and a 95% confidence interval,

Table 7 Comparison of LitCovNet 2's performance on COVID-19-Radigraphy Database with and without CBAM and residual connections

Model	Val Acc	Test Acc	Params
LitCovNet 2	99.26	98.78	67,349
LitCovNet 2 w/o CBAM	98.47	98.42	66,179
LitCovNet 2 w/o Residual	98.97	98.78	67,349

Table 8 Comparison of LitCovNet 2's performance on the COVID-QU-Ex dataset with and without CBAM and residual connections

Model	Val Acc	Test Acc	Params
LitCovNet 2	93.26	95.18	67,349
LitCovNet 2 w/o CBAM	92.99	94.73	66,179
LitCovNet 2 w/o residual	93.17	95.04	67,349

Table 9 Performance summary of LitCovNet 2 evaluated on four clinically realistic variations of the test and validation sets

Evaluation set	Augmentation type	Accuracy (%)
Test	None	98.78
Test	Horizontal flip	98.58
Validation	None	98.97
Validation	Horizontal flip	99.05
Mean		98.85
Standard deviation		0.18
95% confidence interval		[98.46, 99.23]

Reported accuracy values (%) are accompanied by the mean, standard deviation, and 95% confidence interval, highlighting the model's consistent performance

shown in Table 9. The mean accuracy of LitCovNet 2 across all evaluation sets and clinically realistic augmentations is 98.85%, with a low Standard Deviation (SD=0.18) and a 95% Confidence Interval (CI) of [98.46%, 99.23%]. The narrow range of this CI provides strong statistical evidence for the high and reliable performance of this model. Importantly, the fact that minimal degradation in accuracy occurs when testing with the Horizontal Flip augmentation (e.g., 98.78% down to 98.58% on the Test set) demonstrates that LitCovNet 2 maintains high robustness against clinically plausible variations in chest X-ray acquisition. This stability, statistically validated by the tight confidence intervals, indicates that the model is unlikely to exhibit significant performance drop-offs when deployed in diverse clinical environments, which is a key requirement for reliable diagnostic assistance. Given this evaluation setup, the use of k-fold cross-validation is no longer necessary. Cross-validation is typically employed to generate multiple independent performance estimates, especially when model outputs vary due to stochastic training dynamics. However, because our training pipeline is fully deterministic, repeated training across different folds would not introduce meaningful variability. Moreover, the four deterministic yet distributionally distinct evaluation sets, including test and validation in both original and horizontally flipped forms, already provide multiple independent performance measurements without

requiring retraining. This allows us to quantify robustness and compute confidence intervals while retaining the full training set for model optimization, thereby achieving the goals of cross-validation in a more efficient and domain-appropriate manner.

To further assess the interpretability and clinical relevance of LitCovNet 2, we generated Grad-CAM visualizations on the COVID-19 Radiography Database. Representative examples are shown in Fig. 7. The model consistently localized its attention to diagnostically meaningful pulmonary regions associated with infection patterns such as ground-glass opacities and consolidations, rather than spurious background cues. As illustrated by the Grad-CAM overlays and corresponding heatmaps, LitCovNet 2 exhibits stable and class-consistent activation behavior across COVID-19 and viral pneumonia. This alignment between the network's highlighted regions and established radiological findings provides additional evidence that the model is learning clinically relevant features, thereby supporting its reliability for automated chest X-ray classification.

Scaling Up Performance with KD

As we previously discussed, knowledge distillation or KD can be utilized to train a simpler or smaller model with a complex or large model to achieve better performance through the distillation of knowledge which is previously learned by the large model. As we can see from Table 5, LitCovNet 1, being the larger variant, achieved a better test score on the COVID-QU-EX dataset compared to LitCovNet 2 (95.40 and 95.18). This created a scenario where we could take advantage of KD to train LitCovNet 2 with the help of trained LitCovNet 1, where LitCovNet 1 acted as the teacher model and LitCovNet 2 acted as the student model. In the KD experiments, we only utilized the COVID-QU-EX dataset, which is larger and well-balanced, and also quite difficult for the models to achieve a better test accuracy compared to the smaller COVID-19-Radiography dataset.

Though KD allows the opportunity to train a model with a completely different architecture, tuning the hyperparameters like a and t can be quite difficult and tedious. We ran several experiments with various hyperparameter setups and figured out the best values for the optimal results. During the KD process, we use the same processing for the COVID-QU-EX dataset, except changing the batch size down to 64 from 128, which helps to fit the training in the GPU memory. On the optimizer's end, we started with a smaller learning rate of 0.001 and increased the patience of the LR scheduler to 10 while keeping the lowest learning rate to 1e-7, like the previous experiments on the dataset. The training converges within 100 epochs, and when a is set to 0.50 and t is set to 2, the trained student model LitCovNet 2 achieves the test accuracy score of 95.40%, which is the highest score achieved by LitCovNet 1 in our previous experiments. The effectiveness of KD is prominent here as the learned knowledge of the teacher model LitCovNet 1 is properly transferred into the student LitCovNet 2. Through

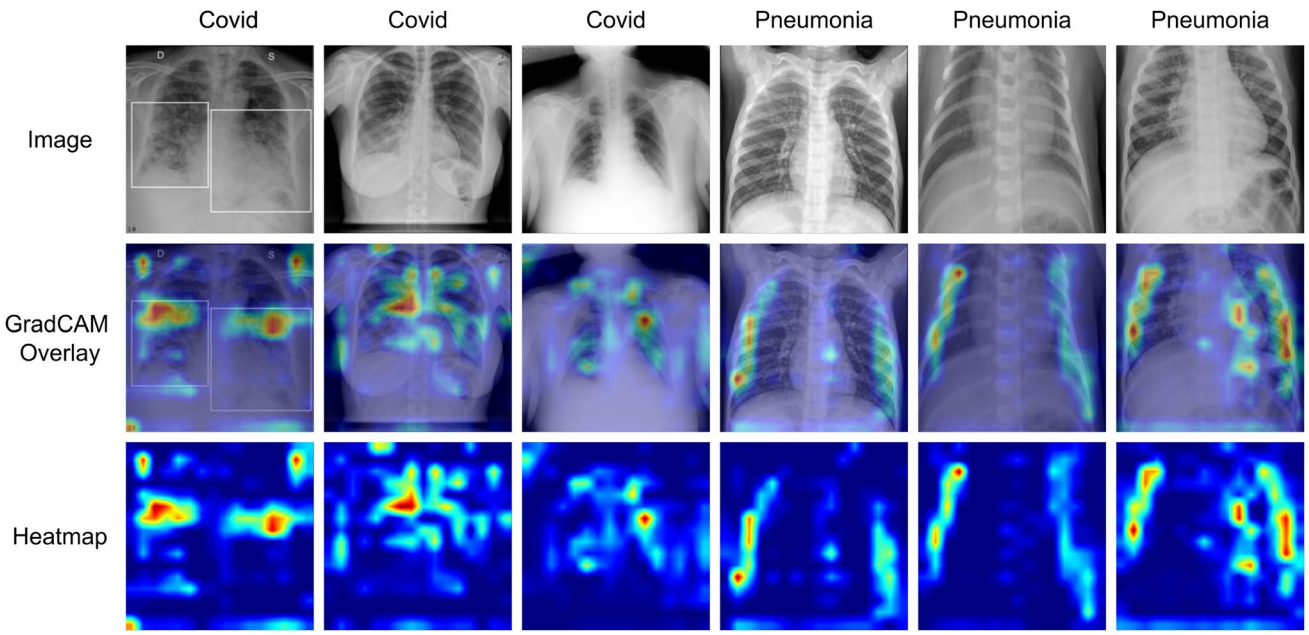

Fig. 7 Visualization of model interpretability for LitCovNet 2 trained on the COVID-19 Radiography Database. The first row shows original chest X-ray images from different classes. The second row presents the corresponding Grad-CAM overlays, highlighting discriminative regions used by the model during prediction. The third row shows the raw Grad-CAM heatmaps. These visualizations demonstrate that LitCovNet 2 consistently attends to clinically relevant pulmonary regions when identifying COVID-19 and other respiratory conditions

Table 10 Details and results of the KD on LitCovNet 1 and LitCovNet 2

Model	Distilled from	KD hyper-params	Test Acc
LitCovNet 2	LitCovNet 1	$\alpha = 0.5, t = 2$	95.40
LitCovNet 2	LitCovNet 1	$\alpha = 0.5, t = 1$	95.64
LitCovNet 1	LitCovNet 1	$\alpha = 0.5, t = 1$	95.70

The table specifies teacher–student configurations, KD hyperparameters (temperature t and balancing factor α), and resulting test accuracies to evaluate the effectiveness of distillation

Table 11 Comparison of recent COVID-QU-Ex classification methods with our proposed LitCovNet models

Work	Method	Accuracy		Params
		Binary (COVID-19 and normal)	Multi-class (COVID-19, Pneumonia and normal)	
Kuzinkovas et al. [41]	Ensemble method	98.34%	94.62%	Not mentioned
Mazari et al. [42]	CNN, ResNet50, MobileNetV2	97.32%	N/A	~3,400,000
Nafisah et al. [43]	CNNs-EfficientNetB7, ResNet50, and MobileNet, Vision Transformers—SegFormer, Twins, and Swin	N/A	87.00% (EfficientNetB7)	~ 66,000,000
Sreena et al. [44]	Pre-trained DenseNet-169	N/A	95.19%	~ 20,242,984
Ahmed et al. [45]	Ensemble model of top performing models from AlexNet, VGG-16, ResNet-18, DenseNet-169, and Inception-V3	N/A	96.002%	Not mentioned
Ibrokhimov et al. [46]	Pre-trained VGG19 and ResNet50	N/A	96.6%	~ 23,500,000
Our work	LitCovNet 1	N/A	95.70%	131,861
Our work	LitCovNet 2	N/A	95.64%	67,349

The table summarizes binary and multiclass accuracy, along with parameter counts, highlighting that LitCovNet 1 and LitCovNet 2 achieve competitive multiclass performance while using significantly fewer parameters than existing CNN-, Transformer-, and ensemble-based approaches

KD, LitCovNet 2 achieved similar test accuracy as its larger variant but only bearing half of the parameters of the larger counterpart. However, lowering the temperature t on the setup lets the student model go beyond this test score as the model achieves a test accuracy score of 95.64%, which beats the score of the complex teacher model. However, when t is set to 1, no softness is applied to the teacher's logits; instead, the direct output of softmax is distilled to the student model. Hence, the student model directly learns to mimic the behavior of the teacher model from the teacher's hard labels. However, in our experiment, this setting helped LitCovNet 2 to achieve higher results than the teacher model itself. The result of all the KD experiments is shown in Table 10.

Inspired by the results achieved by LitCovNet 2's performance after KD, we performed a self-distillation on LitCovNet 1. In self-distillation, the teacher and student are of the same architecture. In this particular experiment, we trained a LitCovNet 1 through KD utilizing another LitCovNet 1 which was already pre-trained on the dataset, and achieved a test score of 95.40%. We use the same hyperparameters that helped LitCovNet 2 to achieve the highest score, where a is set to 0.50 and t is set to 1. Upon finishing the training, the new distilled LitCovNet 1 broke the previous test score performance of LitCovNet 1 and distilled LitCovNet 2, and achieved an accuracy score of 95.70%. It is observed that LitCovNet 1 still stays ahead of its smaller counterpart LitCovNet 2 due to its higher parameter count. However, the final test accuracy performance of the two distilled models is not that far from each other given that the smaller one is almost half the size of the larger model. The test score of distilled LitCovNet 2 is only 0.06% behind that of distilled LitCovNet 1.

Comparative Analysis with Existing Works

In this section, we compare our model's performance with the existing works that were performed on the similar datasets. Though not many works have been performed on these datasets, we compare our results on two of the recent well-established works that provided detailed information on their study and claimed to achieve state-of-the-art results on the corresponding datasets. As seen in Tables 11 and 12, other than our models, the best accuracy score on multiclass classification on the COVID-19 Radiography Database was achieved by the work [23], and the best accuracy score on the COVID-QU-Ex was achieved by the work [46]. Hence, we compare our models' performance with the models mentioned in these works. We compare our models' performance with others by comparing parameter count, floating point operations (FLOPs) required by the models, total size (memory), and test accuracy on the corresponding

Table 12 Comparison of our LitCovNet 2 model with recent works on the COVID-19 Radiography Database

Work	Method	Accuracy binary	Accuracy multi-class	Number of parameters
Ukwandu et al. [21]	MobileNetV2	99.6%	94.5%	3,538,984
Hussein et al. [23]	Lightweight CNN model	98.55%	96.83%	591,903 (Binary) 592,929 (Multiclass)
Our work	LitCovNet 2	N/A	98.78%	67,349

The table reports both binary (COVID-19 vs. Normal) and multi-class (COVID-19, Pneumonia, Normal) accuracy, along with the total number of parameters for each method

Table 13 Quantitative differences between our proposed model and another model that demonstrates the optimization efficiency on the COVID-19-Radiography Dataset

Model	Parameters	GigaFLOPs	Accuracy	Memory (MB)
MobileNetV2	3.5M	0.3190	98.09	13.54
SqueezeNet	1.24M	0.8200	94.72	4.73
LW-CNN	592,929	0.0466	96.83	2.26
LitCovNet 2	67,349	0.7879	98.78	0.2630

Table 14 Quantitative differences between our proposed models against ResNet50, that demonstrates the optimization efficiency on the COVID-QU-Ex Dataset

Model	Parameters	GigaFLOPs	Accuracy	Memory (MB)
ResNet50	23.5M	10.0968	96.6	90.00
LitCovNet 1	131,861	1.3444	95.40	0.5150
Distilled LitCovNet 1	131,861	1.3444	95.70	0.5150
LitCovNet 2	67,349	0.7879	95.18	0.2630
Distilled LitCovNet 2	67,349	0.7879	95.64	0.2630

datasets. To calculate the FLOPs and the size of the models, we needed to recreate the models as mentioned in their corresponding papers and perform an inference operation using a dummy input. To compare our models' performance on the COVID-19-Radiography Database, we implemented the model exactly as mentioned in the work [23]. As the authors did not name their model, for better readability and ease of understanding, we annotate it as Lightweight CNN or LW-CNN. Then we inferred the model with a dummy input of size $112 \times 112 \times 1$, as they worked with single-channel images that are of size 112×112 in resolution. And, to compare our models' performance on the COVID-QU-EX dataset, we loaded a ResNet50 model and dropped the top layer to add just a single dense layer that outputs softmax probabilities for 3 classes. As the work [46] did not mention the details of the trailing layers after the base pre-trained ResNet50 model, we simply added a single output block that corresponds to the output layer. This model was inferred with dummy data that simulates an image of size $256 \times 256 \times 3$, as the authors used 3-channel images of size 256×256. Tables 13 and 14 demonstrate the quantitative differences between our proposed models from the perspective of model optimization compared to other models on similar datasets.

On the COVID-19-Radigraphy dataset, our proposed LitCovNet 2 achieved 1.92% more test accuracy, bearing almost 8.8 times fewer parameters compared to the model mentioned in the work [23]. Due to the lower number of parameters, the memory requirement of the LitCovNet 2 compared to the other models is significantly lower. While the size of the model utilized in the aforementioned work is 2.26 MB, the size of LitCovNet 2 is only 263 KB. However, the other model operates with fewer FLOPS compared to LitCovNet 2, as their model contains a lower number of convolutional filters. On the other hand, on the COVID-QU-EX dataset, our proposed distilled LitCovNet 1 and 2 lag behind only 0.90% and 0.96% in test accuracy scores compared to the ResNet50 model utilized in the work [46]. However, the total parameter count of LitCovNet 1 and 2 is almost 178 times and 350 times lower than that of ResNet50. While the differences in the test accuracy score achieved by our models compared to the large ResNet model achieved by the authors of the paper [46] are marginal, the FLOPs and memory requirements are extremely lower for our proposed models compared to their work. While their model contains around 23.5 million parameters and operates at 10 GigaFLOPS of computational power, our model requires a very low amount of computational power, which is around 1.34 GigaFLOPS for LitCovNet 1 and 0.78 GigaFLOPS for LitCovNet 2. Also, the size of our models is extremely lower compared to the ResNet50 model utilized in the work [46]. While the size of ResNet50 would be at least 90 MB, the size of LitCovNet 1 and 2 is only 515 KB and 263 KB, respectively. The visualization in Fig. 8 demonstrates how small our proposed models are compared to the others. This significant optimization of our models makes our models ideal for deploying in resource-constrained devices where fast and accurate classification is required, but with a very low amount of computational resources.

Performance Benchmark on Edge Devices

To demonstrate the utility and efficiency of such optimized models on resource-constrained devices we chose a Raspberry Pi4B and inferred all 4 models and then compared their performance. The Raspberry Pi4B is a popular ARM-based small single-board computer that costs around 55$ (as of April 2024). It is a popular choice of device for various IoT and edge computing-related research and applications [58,

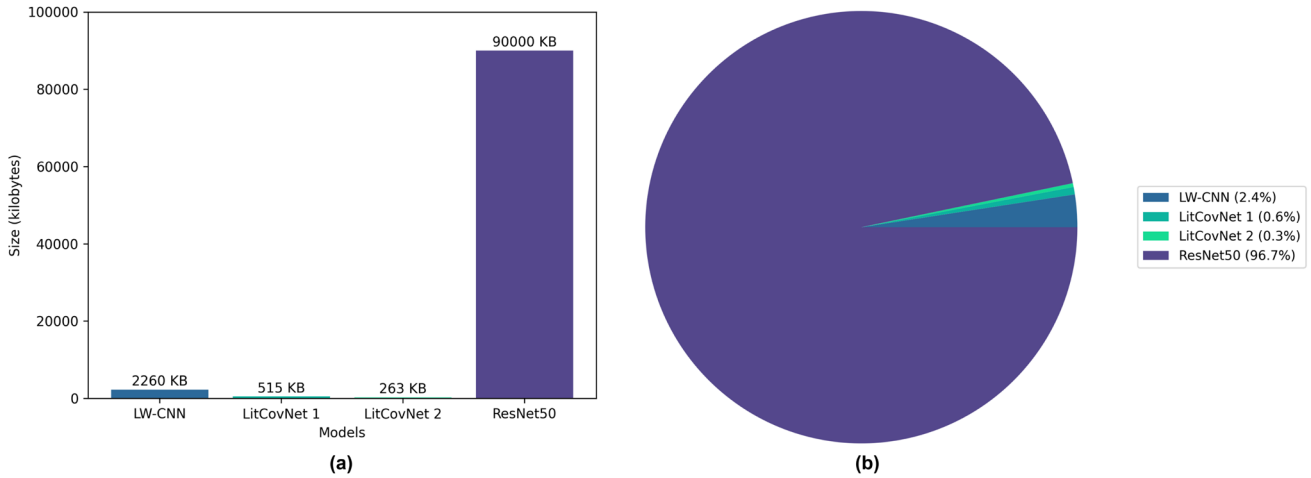

Fig. 8 Memory size comparison visualization among 4 different models. The size of LitCovNet 1 and 2 are significantly smaller than the competitors

59]. The Raspberry Pi4B comes with Broadcom BCM2711 system on a chip (SoC), which is a 64-bit SoC that contains quad-core Cortex-A72, which runs at 1.8 GHz. Our variant contained 4GB LPDDR4-3200 SDRAM and a 128 GB class 10 micro SD card that holds the OS and the codes as the permanent memory. However, the Pi4B does not contain any Neural or Tensor Processing Unit (NPU or TPU) that can aid in ML operations. On the hardware end, we used the complete barebone Pi4B without installing any external cooling solution or case. On the software end, we installed a fresh 64-bit Raspberry Pi OS, which is the official Debian-based Linux distribution for the device. Instead of using the Pi device as a full computer, we connected to the Pi device through SSH and then performed the benchmarks to avoid any other resource requirements by the device.

The main goal of this experiment was to test the performance of our proposed LitCovNet models compared to the two other previously mentioned models, LW-CNN introduced at [23] and ResNet50 introduced at [46]. For this, we created a simple Python script that generates 1000 samples of random data, each sized $256 \times 256 \times 1$, and then infers all the data one by one, and then the average inference time taken by a model to infer all 1000 samples are calculated. Each random data can be thought of as a 256×256-sized 1-channel grayscale image where each pixel contains a float32 value ranging from 0 to 1. Though originally the authors of LW-CNN used $112 \times 112 \times 1$ images, and we previously calculated the parameters and FLOPS according to that, for the benchmarking experiment, we infer the LW-CNN with $256 \times 256 \times 1$-sized data. This increment of the input dimension significantly increases the parameter count, floating-point operation requirement, and the size of the model. The parameter count increased to 2.6 million while the GigaFlops increased to 0.2409, and the size of the model became 10.17 MB. This increment of variables is displayed in Table 15.

Table 15 The differences of parameters, FLOPS requirement, and the size of LW-CNN when initialized for 112×112-sized grayscale images versus 256×256-sized grayscale images

Model	Parameters	GigaFLOPs	Accuracy	Memory (MB)
LW-CNN $(112 \times 112 \times 1)$	592,929	0.0466	96.83	2.26
LW-CNN $(256 \times 256 \times 1)$	2,665,505	0.2409	N/A	10.17

Table 16 Inference-time comparison of lightweight and standard CNN models on the RP4B platform

Model	Inference time (s)
LW-CNN	0.2756
LitCovNet 1	0.3971
LitCovNet 2	0.3808
ResNet50	1.0693

The results show the computational efficiency of the proposed LitCovNet models relative to LW-CNN and ResNet50

As previously mentioned, the benchmark script was run on Raspberry Pi4B through SSH to maximize the utilization of the hardware. For fairness, we ran the benchmark script for each model 5 times, and then we calculated the mean inference time of all the benchmarks. The results of the benchmark on Pi4B are shown in Table 16. Though using $256 \times 256 \times 1$ data increases the parameter count and the size of the LW-CNN, the GigaFLOPS remains the lowest among all the other models. This results in achieving the fastest speed in benchmarking on the Pi device. LitCovNet 2 comes in second followed by LitCovNet 1 and ResNet50. Though the differences between the LitCovNets are very small, the total time taken by the ResNet50 exceeds all the other models. While the 3 other models require around 0.27-−0.39 s to infer a single data, ResNet50 requires slightly

more than 1 s. This makes the ResNet50 model unsuitable for the deployment on edge devices like Raspberry Pi. On the other hand, LW-CNN achieves the best result due to its lower requirement of floating-point operations.

Upon concluding the benchmark on Raspberry Pi4B, we ran the same benchmark procedures on the desktop, which is the experimental setup of the research. Here, the models were run in the 24 GB NVIDIA RTX 4090 GPU, which is the most powerful consumer GPU as of April 2024. This desktop setup can be thought of as a high-end edge device compared to the resource-constrained Raspberry Pi4B. This time, the least amount of time to infer was achieved by LitCovNet 2, which is 0.0346 s. On the other hand, both LitCovNet 1 and LW-CNN took 0.0353 s, resulting in similar time scores despite the fact of having significant differences of parameters and FLOPS. The results are shown in Table 17. The key factor that helped LitCovNets here is the GPU, which performed parallel processing. Hence, even though these models require higher floating-point operations, the parallel processing capability of the GPU aided the models, which resulted in achieving similar or better time scores during inference. However, the ResNet50 stood last among all the models with an average inference time requirement of 0.0532 s. Also the difference in time taken by the best and worst performing model on the Raspberry Pi4B is 0.7937 (1.0693-−0.2756) s, while it is only 0.0186 (0.0532-−0.0346) s on the 4090 GPU. The average inference time required by each model on both Pi4B and RTX 4090 is illustrated in Fig. 9. Also, Fig. 10 shows radar charts for the models inferred on both of the devices, along with other attributes like accuracy, parameters, memory, etc. The radar chart shows the strength of each model for each attribute. Although it is noticeable that hardware like a GPU that aids in ML operations significantly improves the performance of deployed models, along with minimizing gaps in each model's efficiency, having very large parameter count still hinders the process of efficient application of ML models.

Table 17 Inference-time benchmark of the evaluated models on an NVIDIA RTX 4090 GPU

Model	Inference time (s)
LW-CNN	0.0353
LitCovNet 1	0.0353
LitCovNet 2	0.0346
ResNet50	0.0532

The table highlights the speed advantage of the lightweight LitCovNet architectures compared to LW-CNN and the deeper ResNet50

Discussion

In our work, we have followed a scale-up, slim-down approach towards developing optimized models for fast COVID and other respiratory diseases. For this, we first began with a small dataset with a limited training setup and gradually introduced a larger dataset, and then finally slimmed down the model and boosted the performance with knowledge distillation. However, a popular approach for finding optimal networks could be Network Architecture Search or NAS. NAS is an approach that automatically explores various features by searching all possible solutions for optimal architectures. However, NAS has its own drawbacks as it requires a significant amount of computational resources to find the optimal architecture. Also, the black box nature of NAS can lead to less interpretability and explainability of how the model actually works.

Also, hyperparameter tuning is a critical process in any machine learning application. In our experiments it was observed that even the same model might require different

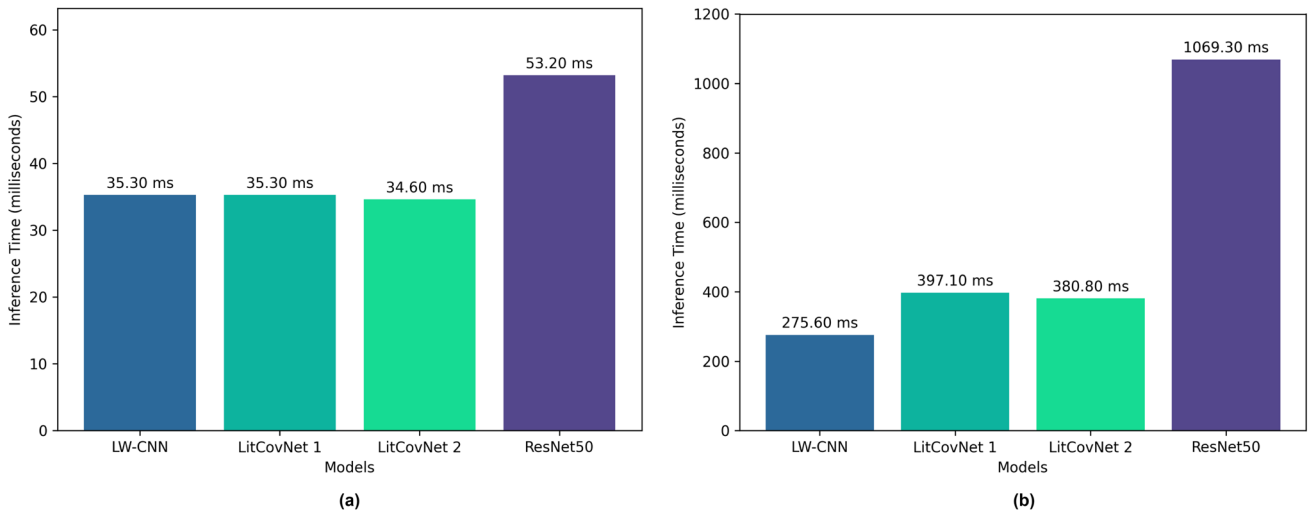

Fig. 9 **a** Bar chart representing the average time required by each model to infer a single data point on the Nvidia RTX 4090 GPU, while **b** represents the similar result on the Raspberry Pi4B

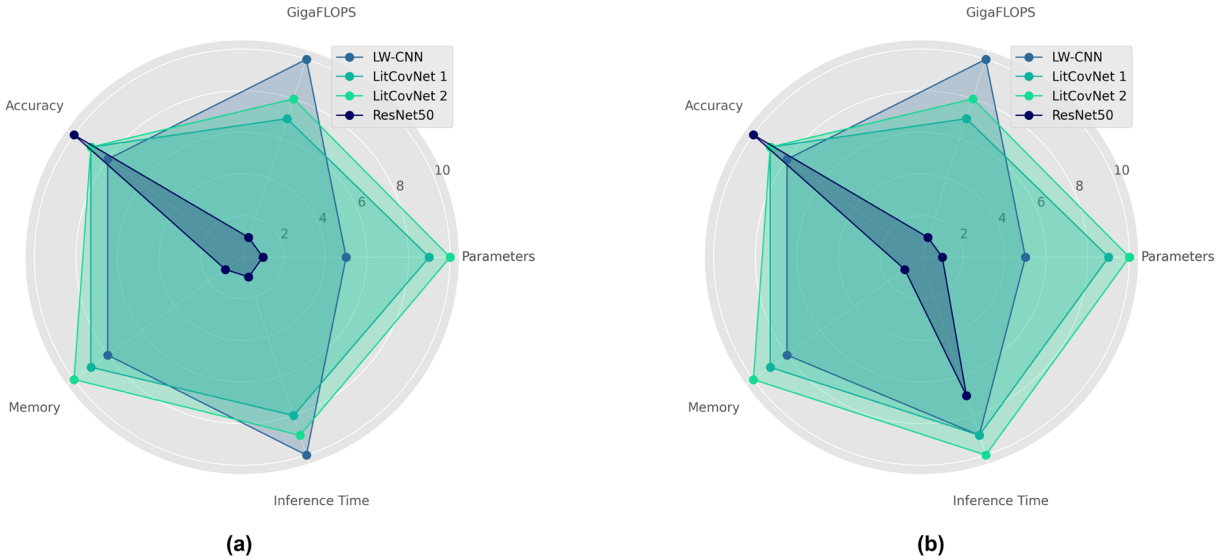

Fig. 10 While the radar chart **a** displays the models' strengths in 5 distinct attributes for Pi4B, the chart **b** does the same for the RTX 4090 GPU. The highest score, which is 10, indicates a model's strength, while the lowest score, which is 0, indicates a model's weakness in a particular attribute. GigaFlops and Accuracy are the strengths of LW-CNN and ResNet50, respectively, while the weaknesses of ResNet50 are Inference Time, Parameters, and Memory. The LitCovNet 2 model's strengths are Inference Time, Parameters, and Memory, while maintaining decent accuracy

hyperparameters to get the optimal results on different datasets. Though in our experiments we tuned the hyperparameters by ourselves through training on several configurations, a better alternative to this approach could be Automated Machine Learning or AutoML. AutoML is an approach to automating machine learning processes. AutoML leverages random search, grid search, and Bayesian optimization to explore various hyperparameters. However, like NAS, AutoML is also an extensive, resource-hungry approach that may require a lot more time and computational resources.

In our experiment we first designed a hybrid CNN that contains both standard convolution operations along with depth-wise separable convolution operations. Also we added residual connection on the smaller model to get the best out of depth-wise separable convolution blocks without increasing the size of parameters. Further we utilized KD to increase the accuracy of the small model with the help of the large model. Here the large model LitCovNet 1 itself was smaller compared to various other common CNN models. Instead of going for a hybrid design, we could utilize only the standard convolutional models that could have achieved more test accuracy, and then, through KD, we could utilize them to increase the accuracy of LitCovNet 2 further. However, in our work, we not only focused on developing optimized models, we also focused on how to train them with low computational cost. For this, we did not go beyond the capabilities of hybrid CNN LitCovNet 1 and did not use a network that contains only standard convolutions, cause this might create a model that is way larger in parameter count and also would required more time and computational power to train and distill other models.

Though we have utilized image augmentation, we used it solely for the purpose of avoiding overfitting of the models during training time. However more image augmentations could be applied to make the models more robust and generalized. Offline augmentations could be used that increases the data count in real folders instead of just randomly applying various augmentation effects on the training batches. Also various X-ray image processing and enhancement techniques could be utilized that could lead to better results. However, we left this for our future work, as here we primarily focused on the optimized network architectures.

We have utilized Knowledge Distillation or KD to optimize and increase the accuracy of our models. Using KD we could significantly increase the accuracy score of the small model LitCovNet 2. However, there are other popular optimizations techniques like quantization and pruning. Both could have been applied to the LitCovNet 2, which could make the model more optimized. However both of these methods require pre- and post-training operations, which are beyond the scope of our current study.

Also, it was observed that dedicated hardware that is efficient in parallel processing significantly increases the performance of the models. Though the GPU used in our research to compare with the CPU of Raspberry Pi4B is not a fair match, the time difference requirement on each platform clearly explains the necessity of ML accelerators in such ML operations. However, this is well understood by the industry, and sophisticated microcomputers with ML

accelerators are being released day by day, which will help build cheap yet efficient ML applications and systems in the future.

Conclusion and Future Works

In this work, we have proposed two lightweight deep learning architectures for fast COVID and other respiratory diseases classification. Our proposed architectures are CNNs consisting of standard convolution, attention mechanism, depth-wise separable convolution, and residual design. Instead of jumping towards utilizing a large-scale popular model, we went through various model optimization methodologies for designing and training the models. Our proposed models, named as LitCovNet 1, contain 131K parameters, and LitCovNet 2 contains only 67K parameters, yet they achieved competitive results on two of the largest multiclass COVID classification datasets. Along with showing the efficacy of hyperparameter tuning, we have utilized Knowledge Distillation to increase the performance of our models. Further we did a comprehensive analysis of our models compared to other models that were utilized in similar datasets to achieve good results. We also benchmarked our models compared to the other models on resource constraint scenarios, and our models performed optimally compared to the other works.

Despite these promising results, the study has a limitation. The CBAM-based attention module used in our design is a generic mechanism that is not specifically tailored to capture radiologically meaningful patterns in medical imaging, potentially limiting its interpretability and effectiveness. As a result, while the model can highlight salient regions, these regions may not always correspond to clinically relevant features, which could affect trust and adoption in real-world diagnostic settings. Future work could explore the integration of domain-specific attention mechanisms or incorporate expert knowledge to enhance both the clinical interpretability and diagnostic performance of the model. Additionally, we aim to conduct comprehensive robustness evaluations to assess model behavior under noise, compression artifacts, and device-specific variations, ensuring consistent and reliable performance in real-world clinical settings.

Acknowledgements This work was supported by the Faculty Research Grant [CTRG-24-SEPS-36], North South University, Bashundhara, Dhaka 1229, Bangladesh.

Author Contributions Conceptualization: MYH, MMHR. Data curation: MMHR Formal analysis: MYH, MMHR. Methodology: MYH, MMHR. Software: MYH, MMHR. Validation: MYH, MMHR. Visualization: MYH, MMHR. Writing-original draft: MYH, MMHR. Writing-review and amp; editing: MYH, MMHR. Project administration: RMR. Supervision: RMR.

Funding Not applicable.

Data Availability Data is publicly available and sources are mentioned in manuscript.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work in this paper.

Informed Consent Not applicable.

Research Involving Human and/or Animals Not applicable.

References

1. Organization WH. Covid-19 dashboard: Cases; 2024, accessed: 2024-02-13 [Online]. Available: https://data.who.int/dashboards/covid19/cases?n=c.
2. Wise J. Covid-19: who declares end of global health emergency; 2023.
3. C. for Disease Control and Prevention, Sars-cov-2 variant classifications and definitions; 2023, accessed: 2024-02-13 [Online]. Available: https://www.cdc.gov/coronavirus/2019-ncov/variants/variant-classifications.html.
4. WebMD, Covid variants; 2023, accessed: 2024-02-13. [Online]. Available: https://www.webmd.com/covid/coronavirus-strains.
5. Litjens G, Kooi T, Bejnordi BE, Setio AAA, Ciompi F, Ghafoorian M, et al. A survey on deep learning in medical image analysis. Med Image Anal. 2017;42:60–88.
6. Shen D, Wu G, Suk H-I. Deep learning in medical image analysis. Annu Rev Biomed Eng. 2017;19:221–48.
7. Chan HP, Samala RK, Hadjiiski LM, Zhou C. Deep learning in medical image analysis. In: Deep learning in medical image analysis: challenges and applications; 2020. p. 3–21.
8. Anthimopoulos M, Christodoulidis S, Ebner L, Christe A, Mougiakakou S. Lung pattern classification for interstitial lung diseases using a deep convolutional neural network. IEEE Trans Med Imaging. 2016;35(5):1207–16.
9. Gargeya R, Leng T. Automated identification of diabetic retinopathy using deep learning. Ophthalmology. 2017;124(7):962–9.
10. Li Y, Shen L. Skin lesion analysis towards melanoma detection using deep learning network. Sensors. 2018;18(2):556.
11. Hu Z, Tang J, Wang Z, Zhang K, Zhang L, Sun Q. Deep learning for image-based cancer detection and diagnosis—a survey. Pattern Recogn. 2018;83:134–49.
12. Wu J, Liu X, Zhang X, He Z, Lv P. Master clinical medical knowledge at certificated-doctor-level with deep learning model. Nat Commun. 2018;9(1):4352.
13. Rajpurkar P, Irvin J, Ball RL, Zhu K, Yang B, Mehta H, et al. Deep learning for chest radiograph diagnosis: a retrospective comparison of the chexnext algorithm to practicing radiologists. PLoS Med. 2018;15(11):e1002686.
14. Jamshidi M, Lalbakhsh A, Talla J, Peroutka Z, Hadjilooei F, Lalbakhsh P, et al. Artificial intelligence and Covid-19: deep learning approaches for diagnosis and treatment. IEEE Access. 2020;8:109581–95.
15. Zhao W, Jiang W, Qiu X. Deep learning for Covid-19 detection based on CT images. Sci Rep. 2021;11(1):14353.
16. Fujita H. AI-based computer-aided diagnosis (AI-CAD): the latest review to read first. Radiol Phys Technol. 2020;13(1):6–19.

17. Kozuka T, Matsukubo Y, Kadoba T, Oda T, Suzuki A, Hyodo T, et al. Efficiency of a computer-aided diagnosis (CAD) system with deep learning in detection of pulmonary nodules on 1-mm-thick images of computed tomography. Jpn J Radiol. 2020;38:1052–61.
18. Gu Y, Chi J, Liu J, Yang L, Zhang B, Yu D, et al. A survey of computer-aided diagnosis of lung nodules from CT scans using deep learning. Comput Biol Med. 2021;137:104806.
19. Abdani SR, Zulkifley MA, Zulkifley NH. A lightweight deep learning model for Covid-19 detection. In: 2020 IEEE symposium on industrial electronics and applications (ISIEA). IEEE; 2020:1–5.
20. Chakraborty M, Dhavale SV, Ingole J. Corona-Nidaan: lightweight deep convolutional neural network for chest X-ray based Covid-19 infection detection. Appl Intell. 2021;51(5):3026–43.
21. Ukwandu O, Hindy H, Ukwandu E. An evaluation of lightweight deep learning techniques in medical imaging for high precision Covid-19 diagnostics. Healthc Anal. 2022;2:100096.
22. Alabbasy FM, Abohamama A, Alrahmawy MF. Compressing medical deep neural network models for edge devices using knowledge distillation. J King Saud Univ Comput Inform Sci. 2023;35(7):101616.
23. Hussein HI, Mohammed AO, Hassan MM, Mstafa RJ. Lightweight deep CNN-based models for early detection of Covid-19 patients from chest X-ray images. Expert Syst Appl. 2023;223:119900.
24. Woo S, Park J, Lee JY, Kweon IS. Cbam: convolutional block attention module. In: Proceedings of the European conference on computer vision (ECCV); 2018. p. 3–19.
25. Sifre L, Mallat S. Rigid-motion scattering for texture classification. Preprint arXiv:1403.1687; 2014.
26. Adegun A, Viriri S. Deep learning techniques for skin lesion analysis and melanoma cancer detection: a survey of state-of-the-art. Artif Intell Rev. 2021;54(2):811–41.
27. Sadad T, Rehman A, Munir A, Saba T, Tariq U, Ayesha N, et al. Brain tumor detection and multi-classification using advanced deep learning techniques. Microsc Res Tech. 2021;84(6):1296–308.
28. Junayed MS, Islam MB, Sadeghzadeh A, Rahman S. Cataractnet: an automated cataract detection system using deep learning for fundus images. IEEE Access. 2021;9:128799–808.
29. Wang W, Xu Y, Gao R, Lu R, Han K, Wu G, et al. Detection of SARS-COV-2 in different types of clinical specimens. JAMA. 2020;323(18):1843–4.
30. Minaee S, Kafieh R, Sonka M, Yazdani S, Soufi GJ. Deep-Covid: Predicting Covid-19 from chest X-ray images using deep transfer learning. Med Image Anal. 2020;65:101794.
31. Dong D, Tang Z, Wang S, Hui H, Gong L, Lu Y, et al. The role of imaging in the detection and management of Covid-19: a review. IEEE Rev Biomed Eng. 2020;14:16–29.
32. Shi F, Wang J, Shi J, Wu Z, Wang Q, Tang Z, et al. Review of artificial intelligence techniques in imaging data acquisition, segmentation, and diagnosis for Covid-19. IEEE Rev Biomed Eng. 2020;14:4–15.
33. Abbas A, Abdelsamea MM, Gaber MM. Classification of Covid-19 in chest X-ray images using DeTraC deep convolutional neural network. Appl Intell. 2021;51:854–64.
34. Sitaula C, Hossain MB. Attention-based VGG-16 model for Covid-19 chest X-ray image classification. Appl Intell. 2021;51(5):2850–63.
35. Shelke A, Inamdar M, Shah V, Tiwari A, Hussain A, Chafekar T, et al. Chest X-ray classification using deep learning for automated Covid-19 screening. SN Comput Sci. 2021;2(4):300.
36. Loey M, Smarandache F, Khalifa NEM. Within the lack of chest Covid-19 X-ray dataset: a novel detection model based on GAN and deep transfer learning. Symmetry. 2020;12(4):651.
37. Waheed A, Goyal M, Gupta D, Khanna A, Al-Turjman F, Pinheiro PR. Covidgan: data augmentation using auxiliary classifier GAN for improved Covid-19 detection. IEEE Access. 2020;8:91916–23.
38. Tanenbaum LN, Tsiouris AJ, Johnson AN, Naidich TP, DeLano MC, Melhem ER, et al. Synthetic MRI for clinical neuroimaging: results of the magnetic resonance image compilation (magic) prospective, multicenter, multireader trial. Am J Neuroradiol. 2017;38(6):1103–10.
39. Rahman T, Khandakar A, Qiblawey Y, Tahir A, Kiranyaz S, Kashem SBA, et al. Exploring the effect of image enhancement techniques on Covid-19 detection using chest X-ray images. Comput Biol Med. 2021;132:104319.
40. Tahir AM, Chowdhury ME, Khandakar A, Rahman T, Qiblawey Y, Khurshid U, et al. Covid-19 infection localization and severity grading from chest X-ray images. Comput Biol Med. 2021;139:105002.
41. Kuzinkovas D, Clement S. The detection of Covid-19 in chest X-rays using ensemble CNN techniques. Information. 2023;14(7):370.
42. Mazari AC, Kheddar H, Deep learning-and transfer learning-based models for Covid-19 detection using radiography images. In: 2023 international conference on advances in electronics, control and communication systems (ICAECCS). IEEE; 2023. p. 1–4.
43. Nafisah SI, Muhammad G, Hossain MS, AlQahtani SA. A comparative evaluation between convolutional neural networks and vision transformers for Covid-19 detection. Mathematics. 2023;11(6):1489.
44. Sreena V, Ponraj DN. Optimization of fine tuned deep learning model for multiclass classification of chest X-rays. In: 2023 4th international conference on signal processing and communication (ICSPC). IEEE; 2023, p. 173–7.
45. Ahmed ST, Mahdin MF, Barua S, Abir FS, Fahim-Ul-Islam M, Chakrabarty A. Utilizing deep learning architectures for early detection of lung diseases in chest X-ray images. In: 2023 26th international conference on computer and information technology (ICCIT). IEEE; 2023. p. 1–6.
46. Ibrokhimov B, Kang J-Y. Deep learning model for Covid-19-infected pneumonia diagnosis using chest radiography images. BioMedInformatics. 2022;2(4):654–70.
47. He K, Zhang X, Ren S, Sun J, Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition; 2016. p. 770–8.
48. Cheng Y, Wang D, Zhou P, Zhang T, A survey of model compression and acceleration for deep neural networks. Preprint arXiv:1710.09282; 2017.
49. Deng S, Zhao H, Fang W, Yin J, Dustdar S, Zomaya AY. Edge intelligence: the confluence of edge computing and artificial intelligence. IEEE Internet Things J. 2020;7(8):7457–69.
50. Tahir AM, Chowdhury MEH, Qiblawey Y, Khandakar A, Rahman T, Kiranyaz S, Khurshid U, Ibtehaz N, Mahmud S, Ezeddin M. "Covid-qu-ex," Kaggle, 2021, accessed: 2024-03-10 [Online]. Available: https://doi.org/10.34740/kaggle/dsv/3122898.
51. LeCun Y, Bottou L, Bengio Y, Haffner P. Gradient-based learning applied to document recognition. Proc IEEE. 1998;86(11):2278–324.
52. Krizhevsky A, Sutskever I, Hinton GE. ImageNet classification with deep convolutional neural networks. Adv Neural Inform Process Syst. 2012;25:1.
53. Lin M, Chen Q, Yan S, Network in network. Preprint arXiv:1312.4400; 2013.
54. Chollet F, Xception: deep learning with depthwise separable convolutions. In: Proceedings of the IEEE conference on computer vision and pattern recognition; 2017. p. 1251–8.
55. Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, Andreetto M, Adam H. Mobilenets: efficient convolutional

neural networks for mobile vision applications. Preprint arXiv:1704.04861; 2017.
56. Hinton G, Vinyals O, Dean J, Distilling the knowledge in a neural network. Preprint arXiv:1503.02531; 2015.
57. Heidari M, Mirniaharikandehei S, Khuzani AZ, Danala G, Qiu Y, Zheng B. Improving the performance of CNN to predict the likelihood of Covid-19 using chest X-ray images with preprocessing algorithms. Int J Med Inform. 2020;144:104284.
58. Johnston SJ, Cox SJ, The raspberry Pi: a technology disrupter, and the enabler of dreams; 2017. p. 51.
59. Jolles JW. Broad-scale applications of the raspberry Pi: a review and guide for biologists. Methods Ecol Evol. 2021;12(9):1562–79.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.